

Env2FSA: Automatic Symbolic Environment Modeling for Behavioral Controller Verification

Tomoki Kuzuhara*, Senling Wang*, Kerun Li, Shintaro Ono, Hiroshi Kai, Hiroshi Takahashi

*Graduate School of Science and Engineering, Ehime University, Japan

n807012x@mails.cc.ehime-u.ac.jp, wang@cs.ehime-u.ac.jp

Abstract— In this paper, we present *Env2FSA*, a framework that automatically constructs scene-specific symbolic environment models for the formal verification of robot behavior planning. The proposed method uses waypoint images and a typed PDDL task description as inputs to extract scene-dependent grounded propositions and action-transition conditions, which it then translates into a NuSMV environment model. By combining visual grounding with PDDL-consistent symbolic modeling, *Env2FSA* reduces the manual effort required to prepare environment models for verification-based, high-level planning across diverse operating environments. The generated models are integrated with *GLM2FSA*, which synthesizes finite-state controllers from natural-language task descriptions. The models are then refined using counterexample-guided interaction. Experiments in Virtual Home-based simulated scenarios provide preliminary evidence that the proposed framework produces valid symbolic actions and supports successful, verified task execution after refinement. These results suggest that automatic environment model construction is a practical step toward scalable, formal verification of scene-dependent robot behavior planning.

Keywords—symbolic environment model, visual grounding, formal verification, controller synthesis, linear temporal logic, household task planning, visual-language-model

I. INTRODUCTION

Recent advances in large language models have accelerated research on generating robot action plans from natural-language task instructions. Prior studies have explored affordance-aware action selection [4], demonstrating the potential of language-based robot planning. However, many of these approaches generate plausible action sequences without rigorously verifying whether the resulting behavior satisfies environmental transitions and safety constraints. In this context, *GLM2FSA* [1] is important because it translates a natural-language task instruction into a finite-state action controller and enables model checking against an environment model and temporal logic constraints [6], [7]. Thus, *GLM2FSA* makes robot behavior generation from language mathematically verifiable.

However, applying *GLM2FSA* to a concrete scene still depends on a manually constructed symbolic environment model that defines propositions, actions, and state-transition relations. Related studies have explored symbolic model construction from multimodal observations, such as PDDL problem generation for planning [5] and online grounding of symbolic planning domains [8]. Yet, these methods are mainly intended for planning support rather than for constructing a verification-ready environment transition model. To address this issue, we propose *Env2FSA*, which automatically generates scene-specific symbolic environment models from waypoint images and typed PDDL descriptions. [2]. The *Env2FSA*'s key innovation is its ability to directly generate a verification-ready transition model

from scene observations under typed symbolic constraints. *Env2FSA* formally verifies planning across diverse environments by synthesizing scene observations and symbolic environment models.

II. PROBLEM FORMULATION AND PROPOSED METHOD

A. Problem Formulation

Given waypoint observation images I , a typed PDDL domain D , an initial symbolic state s_0 , a natural-language task description t , and an LTL specification φ , the goal of this work is to automatically construct a scene-specific symbolic environment model and synthesize a formally verified controller for the target task [1], [2], [7].

The environment model is represented as $M_E = \langle P, A, T, s_0 \rangle$, where P denotes the set of grounded environment propositions, A denotes the set of grounded action propositions, T is the transition relation, and s_0 is the initial environment state.

The action controller is represented as $M_C = \langle Q, q_0, \delta, \lambda \rangle$, where Q is the set of controller states, q_0 is the initial controller state, δ is the transition function, and λ is the output labeling function. The objective is to generate M_E automatically from scene observations and typed PDDL, and then combine it with the controller generated from t so that the closed-loop behavior satisfies φ [1], [2], [6], [7].

In our framework, the controller generation is handled by a *GLM2FSA*-style synthesizer [1], while the focus of *Env2FSA* is the automatic construction of M_E . The generated environment model is later used for model checking and counterexample-guided controller refinement [1], [6].

B. Overview of the Proposed Method

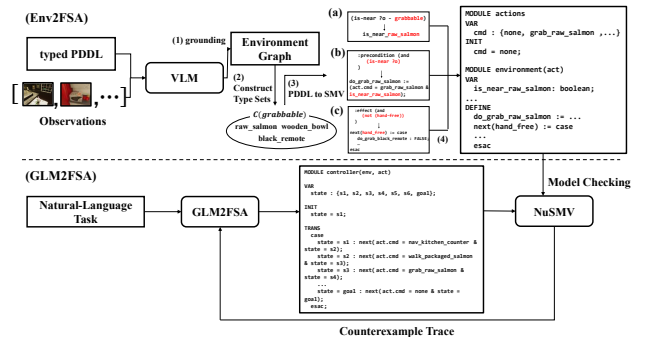


Fig. 1. Overall pipeline of *Env2FSA*, from waypoint observations and typed PDDL to environment-model construction, controller verification, and counterexample-guided refinement in NuSMV.

The overall pipeline is shown in **Fig. 1**. Given waypoint images I and a typed PDDL description D , *Env2FSA* first

grounds scene elements using a vision-language model and organizes them into an environment graph. It then constructs grounded type-consistent propositions and actions based on the PDDL type system, and translates their preconditions and effects into a NuSMV environment model M_E . This environment model is combined with the controller M_C generated by GLM2FSA and used for model checking against the LTL specification.

This study's main idea is to use typed PDDL as an intermediate symbolic specification that constrains scene grounding and transition construction. This approach ensures that the generated environment model remains consistent with the symbolic task description while reflecting the observed scene.

C. Required Definitions in the Typed PDDL Domain

```
(define (domain virtualhome)
  (:types
    can_open grabbable has_switch surface - object
    waypoint
  )
  ;; can_open: can be opened
  ...
  (:predicates
    (is-near ?o - object) ;; near o
  )
  ...
  ;; Robot constraints
  ;; Only objects within 1 meter can be manipulated.

  ;; move to waypoint wp
  (:action nav
    :parameters (?wp - waypoint)
    :precondition (and (is-near ?o) (not (is-sealed ?o)))
    :effect (and (not (hand-free)))
  )
  ...
)
```

Fig. 2. Example of a typed PDDL domain description, including types, predicates, actions, and explanatory comments.

Fig. 2 shows an example of a PDDL (Planning Domain Description Language) domain description used in the proposed framework. In our framework, a PDDL domain description must be manually prepared in advance. In addition to types, predicates, and actions, it is also required to define comments that explain their semantics and the agent's capabilities. These comments play an essential role in enabling the VLM to understand the context of the target environment and to correctly ground affordances to relevant objects. Specifically, the PDDL description requires three categories of types: "object," "waypoint," and affordance-related types. Their roles are as follows:

- "object": arbitrary objects in the environment.
- "waypoint": the names of waypoints.
- affordance-related: the affordances of individual objects, such as "surface" and "grabbable".

Here, "object" is defined as the parent type of affordance-related types. Predicates represent observable states of the environment, while actions represent executable behaviors of the agent, including their preconditions and effects. The argument types of predicates and actions are defined to specify the affordances required for each object argument. Providing these definitions in advance allows the framework to appropriately ground affordance-annotated objects into valid predicates and actions. Additionally, comments describing the roles of types, predicates, and

actions, as well as the agent's capabilities when acting on the environment, help the VLM determine which affordances should be assigned to which objects.

D. Construction of the Environment Graph and Type Sets

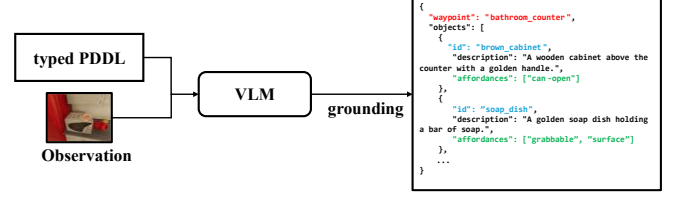


Fig. 3. Example of generating a JSON-formatted entry of the environment graph G from a single waypoint image and a typed PDDL description using a vision-language model (VLM).

As shown in Fig. 3, the environment graph is generated from a typed PDDL domain D and waypoint images I . The waypoint images are input to the vision-language model (VLM) one by one, and the extracted objects, affordances, and waypoints are finally integrated into a single list, which constitutes the environment graph G in JSON format.

Next, a candidate set is constructed for each type τ from the obtained environment graph G . The candidate set is defined as follows:

$$C(\tau) = \begin{cases} O & (\tau = \text{object}), \\ W & (\tau = \text{waypoint}), \\ \{o \in O \mid \tau \in \text{aff}(o)\} & (\text{otherwise}), \end{cases} \quad (1)$$

where O denotes the set of all objects, W denotes the set of waypoints, and $\text{aff}(o)$ denotes the function that returns the set of affordances grounded to object o . Using these candidate sets, objects are grounded into valid predicates and actions, and environment propositions and action propositions are then generated.

E. SMV Encoding of the Environment Model M_E

In Env2FSA, the environment model is represented as $M_E = \langle P, A, T, s_0 \rangle$, where P is the set of grounded environment propositions, A is the set of grounded action propositions, T is the transition relation, and s_0 is the initial state. In this section, we describe how these components are encoded in the SMV language of NuSMV to construct the verification-ready environment model M_E .

First, the grounded environment propositions in P are generated from the predicates declarations of the typed PDDL description. For a predicate $f(\tau_1, \dots, \tau_n)$, Env2FSA enumerates the Cartesian product $C(\tau_1) \times \dots \times C(\tau_n)$ and generates one grounded proposition for each tuple. For example, "on(?o1 - grabbable ?o2 - surface)" yields propositions such as "on_apple_table". In the SMV model, each proposition $p \in P$ is represented as a Boolean state variable in the "environment" module. Zero-argument predicates are directly represented as single Boolean variables.

Next, the grounded action propositions in A are generated from the action schemas in the same manner, by grounding the Cartesian product of the candidate sets corresponding to the parameter types. Each grounded action is then represented in SMV as one symbol in the enumerated command variable "act.cmd" in the "actions" module. Thus, the set A is encoded as the action alphabet that the environment model can observe and respond to.

The transition relation T is encoded by translating the grounded preconditions and effects of each action proposition into SMV expressions. For each grounded action proposition a , its executability condition is defined as

$$do_a = (act.cmd = a \ \& \ \langle precondition \rangle) \quad (2)$$

where $\langle precondition \rangle$ denotes the grounded SMV logical formula obtained from the PDDL precondition. Thus, do_a becomes true only when action a is selected and its precondition is satisfied.

Then, the effects of all grounded actions are compiled into the “next” definitions of the environment propositions. Let $Eff(a) \subseteq P$ denote the set of propositions affected by action a , and let

$$A_p = \{a \in A \mid p \in Eff(a)\} \quad (3)$$

be the set of actions whose effects include proposition p . Then, for each $p \in P$, the transition relation is encoded in SMV as

$$\forall p \in P, \quad next(p) := \begin{cases} v_1 & (\text{if } do_a_1 \wedge (a_1 \text{ assigns } p := v_1)) \\ v_2 & (\text{if } do_a_2 \wedge (a_2 \text{ assigns } p := v_2)) \\ \vdots & \\ p & (\text{otherwise}) \end{cases}$$

$$a_i \in A_p, \quad v_i \in \{\text{TRUE}, \text{FALSE}\} \quad (4)$$

This means that T is realized in SMV by aggregating, for each proposition p , all grounded actions that may update it. If one of their conditions holds, p transitions to the value specified by the corresponding effect; otherwise, its current value is preserved. For example, if proposition p is set to “TRUE” by action a_1 and to “FALSE” by action a_2 , its SMV description becomes

```
next(p) := case
  do_a_1 : TRUE;
  do_a_2 : FALSE;
  TRUE : p;
esac
```

Finally, the initial state s_0 is encoded using the “init” construct of SMV. For each proposition $p \in P$, “init(p)” is set to “TRUE” if $p \in s_0$, and to “FALSE” otherwise.

In the NuSMV model, “cmd” is defined in the “actions” module, while the executability conditions, transition definitions, and initial-value definitions are described in the “environment(act)” module together with the Boolean environment propositions. In this way, the four components P , A , T , and s_0 are described in SMV and together constitute the NuSMV environment model M_E .

F. Controller Synthesis and Model Checking

First, GLM2FSA uses an LLM to convert a task instruction given in natural language into a sequence of natural-language action steps. Then, it performs grammatical analysis on the resulting step sequence to extract the transition condition and the action corresponding to the output symbol from each step. Treating each step as a single state allows GLM2FSA to construct a finite-state automaton (FSA) and obtain an action controller M_C represented in FSA form from the natural-language task instruction.

The generated action controller M_C is then model-checked in NuSMV together with the environment model M_E automatically constructed by Env2FSA and the constraint ϕ , by verifying whether the composed system $M_E \parallel M_C$ satisfies ϕ . If the generated controller is inconsistent with the environment transitions or violates the given constraint, NuSMV returns a counterexample trace showing the violation. In our framework, this counterexample trace is appended to the LLM prompt, and the action controller M_C is regenerated. By iterating this closed-loop process of controller generation, model checking, and counterexample-guided regeneration, the framework finally obtains an action controller that satisfies both the environment transitions and the specified constraint ϕ .

III. EXPERIMENTAL SETTINGS

The experiments were conducted on household scenes in VirtualHome [9] using waypoint observation images, a typed PDDL domain, an initial symbolic state, natural-language task instructions, and LTL specifications representing task goals and safety constraints. Env2FSA was used to automatically construct the symbolic environment model from the images and typed PDDL, and the resulting model was combined with a GLM2FSA-style controller and verified in NuSMV. For both environment grounding and controller generation, we used gemini-2.5-flash as the foundation model.

The evaluated task categories were:

1. *fetch-and-deliver*: a task in which a target object is carried to a designated place
2. *microwave use with a safety constraint*: a task involving microwave operation under a safety constraint
3. *pick-and-place across rooms*: a task that requires moving an object between rooms
4. *switch-dependent routines*: tasks whose execution depends on the state of switches

For example, in the *fetch-and-deliver* category, we set the task instruction to “Bring the black_remote to the kitchen counter” and the LTL specifications to $F \text{ on_black_remote_kitchen_counter} \wedge G(\text{grab_black_remote} \rightarrow \text{is_near_black_remote})$.

To evaluate generality, we prepared three different scenes for each task category, and each scene was represented by four waypoint observations. In total, one task instance was assigned to each scene, resulting in 12 task instances. As baselines, we compared Manual + GLM2FSA, Env2FSA (initial only), and Env2FSA + CE-refine.

IV. EVALUATION METRICS

The evaluation metrics were divided into **scene-level metrics** for environment-model quality and **task-level metrics** for controller synthesis, verification, and execution. Let S be the set of scenes and $\mathcal{T}$ be the set of tasks; in our experiments, $|S| = 3$, $|\mathcal{T}| = 12$. For each scene $s \in S$, let P_s^{pred} be the set of automatically generated propositions and P_s^{ref} be the proposition set of the manually constructed reference model. Let A_s^{pred} be the generated action set and A_s^{valid} the valid action set under the typed PDDL.

Proposition-level agreement was evaluated by Proposition F1. For each scene, precision and recall are defined as

$$Prec_s = \frac{|p_s^{pred} \cap p_s^{ref}|}{|p_s^{pred}|}, Rec_s = \frac{|p_s^{pred} \cap p_s^{ref}|}{|p_s^{ref}|} \quad (5)$$

and their harmonic mean is denoted by $F1_s$. The overall Proposition F1 is then computed as

$$PropF1 = \frac{1}{|S|} \sum_{s \in S} F1_s \quad (6)$$

To evaluate whether action grounding was consistent with affordances and type constraints, we define the Valid Action Grounding Ratio as

$$Valid = \frac{1}{|S|} \sum_{s \in S} \frac{|A_s^{ref}|}{|A_s^{pred}|} \quad (7)$$

For each task instance $i \in \mathcal{T}$, let $l_i^{(0)} \in \{0,1\}$ denote whether the initial controller satisfies the LTL specification, $l_i^{(f)} \in \{0,1\}$ whether the final controller after counterexample-guided regeneration satisfies it, and $u_i^{(f)} \in \{0,1\}$ whether the final controller succeeds in VirtualHome execution. Let $t_i^{(f)}$ be the total runtime including refinement and verification. From these values, we report the initial LTL satisfaction rate, final LTL satisfaction rate, final task success rate, and average runtime.

V. RESULTS AND DISCUSSION

Table I summarizes the quantitative results for both environment-model quality and task performance over the evaluated scenes and task instances. The initially generated controllers were not sufficiently reliable: Initial-LTL was 0.58 for *Manual + GLM2FSA* and 0.67 for both *Env2FSA (initial only)* and *Env2FSA + CE-refine*. A major cause of failure was that some generated controllers selected the next action before its local precondition was satisfied, which led to task failures during execution. However, counterexample-guided regeneration was effective; both *Manual + GLM2FSA* and *Env2FSA + CE-refine* finally achieved 1.00 in both Final-LTL and Final-Task.

For the environment model itself, PropF1 was 0.568, while Valid was 1.00. The perfect Valid score indicates that grounded affordances were consistent with the typed PDDL constraints. In contrast, the lower PropF1 mainly came from reduced precision: although the VLM was expected to exclude distant objects, many objects that did not satisfy the intended distance condition were still extracted. As a result, too many non-target objects were included, unnecessary propositions increased, and precision decreased. **Fig. 4** shows a representative waypoint image and its corresponding environment graph, illustrating that non-target objects were also extracted and converted into propositions, which contributed to the lower precision.

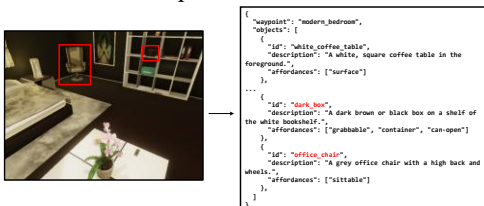


Fig. 4. Example of over-extraction in environment model generation.

TABLE I Comparison of environment modeling and task performance over 3 scenes and 12 task instances.

Method	PropF1	Valid	Init-LTL	Final-LTL	Final-Task	Runtime [s]:
<i>Manual+GLM2FSA</i>	N/A	N/A	0.58	1.00	1.00	1317.2
<i>Env2FSA (initial only)</i>	0.57	1.00	0.67	0.67	0.25	102.8
<i>Env2FSA + CE-refine</i>	0.57	1.00	0.67	1.00	1.00	1407.3

Table I also shows that runtime remains a major limit. *Manual + GLM2FSA* required 1317.2 s, *Env2FSA (initial only)* required 102.8 s, and *Env2FSA + CE-refine* required 1407.3 s. Thus, in the successful settings, verification took about 20 minutes. This result highlights the need to reduce the size of the NuSMV model by selecting only the propositions necessary for each task, rather than including many irrelevant atomic propositions. Another current limitation is that the model does not yet support external entities whose states change autonomously; that is, spontaneous transitions are not modeled.

VI. CONCLUSION

We presented *Env2FSA*, a method for automatically generating symbolic environment models from waypoint images for LTL-based controller synthesis and verification. The results showed that the generated models were consistent with typed PDDL constraints and, with counterexample-guided refinement, supported perfect final task success and LTL satisfaction in the evaluated tasks.

ACKNOWLEDGMENT

This research was supported by TAIYO YUDEN CO., LTD. The author would like to sincerely thank Mr. Hiromoto Nagamine, Mr. Mark Hozoji and Miss Rie Naito of TAIYO YUDEN CO., LTD. for their valuable advice and technical support.

REFERENCES

- [1] Y. Yang, C. Neary, and U. Topcu, "Automaton-Based Representations of Task Knowledge from Generative Language Models," in *Proc. Int. Conf. Neuro-symbolic Syst.*, vol. 288, *Proc. Mach. Learn. Res. (PMLR)*, 2025, pp. 765–783.
- [2] M. Ghallab, A. Howe, C. Knoblock, D. McDermott, A. Ram, M. Veloso, D. Weld, and D. Wilkins, "PDDL—The Planning Domain Definition Language, Version 1.2," Yale Center for Computational Vision and Control, Tech. Rep. CVC TR-98-003/DCS TR-1165, Oct. 1998.
- [3] A. Cimatti, E. M. Clarke, F. Giunchiglia, and M. Roveri, "NuSMV: A New Symbolic Model Checker," *Int. J. Softw. Tools Technol. Transfer*, vol. 2, no. 4, pp. 410–425, 2000, doi: 10.1007/s100090050046.
- [4] B. Ichter *et al.*, "Do As I Can, Not As I Say: Grounding Language in Robotic Affordances," in *Proc. 6th Conf. Robot Learn. (CoRL)*, vol. 205, *Proc. Mach. Learn. Res.*, 2023, pp. 287–318.
- [5] K. Shirai, C. C. Beltran-Hernandez, M. Hamaya, A. Hashimoto, S. Tanaka, K. Kawaharazuka, K. Tanaka, Y. Ushiku, and S. Mori, "Vision-Language Interpreter for Robot Task Planning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024, pp. 2051–2058.
- [6] E. M. Clarke, E. A. Emerson, and A. P. Sistla, "Automatic verification of finite-state concurrent systems using temporal logic specifications," *ACM Transactions on Programming Languages and Systems*, vol. 8, no. 2, pp. 244–263, Apr. 1986, doi: 10.1145/5397.5399.
- [7] A. Pnueli, "The Temporal Logic of Programs," in *Proc. 18th Annu. Symp. Found. Comput. Sci. (FOCS)*, pp. 46–57, doi: 10.1109/SFCS.1977.32.
- [8] L. Lamanna, L. Serafini, A. Saetti, A. Gerevini, and P. Traverso, "Online Grounding of Symbolic Planning Domains in Unknown Environments," in *Proc. 19th Int. Conf. Principles Knowl. Representation Reasoning (KR)*, 2022, pp. 511–521.
- [9] X. Puig, K. Ra, M. Boben, J. Li, T. Wang, S. Fidler, and A. Torralba, "VirtualHome: Simulating Household Activities via Programs," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 8494–85